

全局 coflow：32 NPU 与 5/6/7 SSU

同一可变输入、5 ms 遥测、六策略完整对照

2026-09-07

Contents

1 先看结论与适用范围	1
2 主结果：种子 20260906	2
3 泛化检查：种子 20260907	4
4 如何选择参数，以及没选什么	6
5 输入究竟是什么	6
6 三个新增策略究竟在哪一侧起作用	7
6.1 S1：在排进 SSD 以前协调	7
6.2 S2：已入盘后，仅在原生等价 Path 中选	8
6.3 S3：同样的可选范围，用全局 coflow 进度键	8
7 为什么协调更多也未必消除等待	8
7.1 数学证书和原生仿真到底匹配什么	8
8 全程 stall 与尾部不均衡：不能只看中间一秒	9
9 控制代价与部署限制	11
10 函数、归档与复现	11

1 先看结论与适用范围

36 组正式对照已完成。Baseline 在六个拓扑/种子组合中的固定秒 U 为 83.78%-99.11%。以下同时报告三个新策略的收益与退化，不只展示最好的一个点。

用户目标目前只部分达到：S3 相对 Once 的两种子平均 U，6 盘改变 +2.43 个百分点，5 盘改变 -2.97 个百分点（明显退化），7 盘仅 +0.28 个百分点。三个新策略在每个拓扑/种子组合的完整 makespan 都比 Once 更长，未验证到整批完成吞吐提升。接纳 SLO 改善与窗口 U/整批时长的取舍必须同时保留，不能称为全面胜出。

策略	U 提高组数	U 变化/pp	接纳 SLO 提高	全程更快
S1	5/6	-5.33...+3.89	6/6	0/6
S2	3/6	-5.13...+3.52	6/6	0/6
S3	4/6	-5.59...+4.24	6/6	0/6

上表相对 Once，变化范围单位为百分点；全程更快以完整 makespan 判定。能力更多不能直接等同所有输入都更好，也不保证无 stall。

本轮不是旧独立 JIT 的改名：S1 新增全局入队前发射协调，S2/S3 才增加盘内已排队命令的选择权限。完整 36 组均单独运行原生仿真器；不从旧轮搬结果，不按测试种子重新选参数。开发只使用 6 盘、种子 20260906；另一个种子和其他拓扑是泛化检查，不是独立线上分布的保证。

这不是 218.31565 GiB/s 对三种拓扑都“长期容量可行”的输入。后续用户请求携带的八层工作率保持相同，已超过 5 盘总计 200 GiB/s。若此到达率持续，重排不能防止总积压增长；但本次 704 请求是有限输入，仍能排空，不

能仅由这个平均值推出固定 [1,2] 秒 U 上限或全部 SLO 必失败。6/7 盘则仍有局部 deadline、跨请求预取预算和调度成本约束。初始 128 请求积压另算，不伪装为后续平均速率的一部分。

主 U：所有 32 卡固定 [1000,2000] ms 的计算时间占比平均。主 SLO：同一完整 704 个请求中 completion-admission $\leq 1.5 \times 8C$ ，其中 C 是该请求每层的纯计算时间，8C 是八层纯计算总时间；此 SLO 从接纳起算。用户 SLO 从 arrival 起算，还包括接纳前排队。两种口径不能互换；主 SLO 提高不自动意味着用户体验同幅提高。

请求参数出自项目 data，但配额、顺序与到达时钟是构造的。它是可复现的变长请求机制实验，不是生产到达分布采样。所有额外控制计算和跨客户端协调延迟均未计入仿真时间，部署收益仍须单独验证。

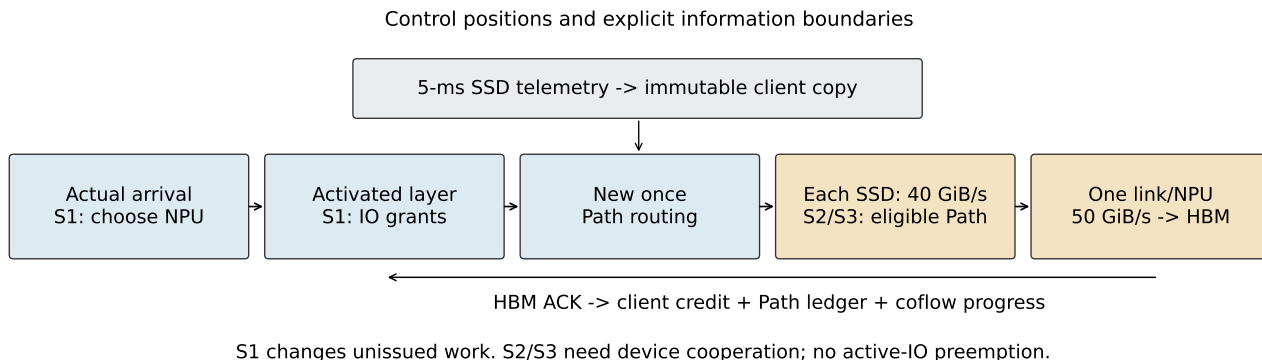


Figure 1：控制位置示意。图中主流程是 S1-S3；Baseline/Once 不启用新选卡与全局发射。所有策略都使用同一个周期 collector，只有主机副本可查询；S2/S3 的设备本地已排队信息是另一项明确增加的权限。下方箭头反馈是客户端 HBM ACK，不是额外实时读 SSD。

2 主结果：种子 20260906

SSU	策略	U/%	接纳 SLO	用户 SLO	active
5	Baseline	83.78	448/704	23/704	32
5	Once/layer	83.99	568/704	28/704	32
5	New once	84.14	553/704	30/704	32
5	S1	84.30	651/704	24/704	32
5	S2	83.30	638/704	25/704	32
5	S3	83.63	651/704	28/704	32
6	Baseline	93.26	555/704	27/704	32
6	Once/layer	93.74	663/704	35/704	32
6	New once	94.64	668/704	35/704	32
6	S1	97.63	693/704	36/704	32
6	S2	97.26	690/704	35/704	32
6	S3	97.99	690/704	36/704	32
7	Baseline	98.47	615/704	33/704	32
7	Once/layer	99.14	690/704	41/704	32
7	New once	99.29	691/704	44/704	32
7	S1	99.56	695/704	41/704	32
7	S2	99.73	694/704	40/704	32
7	S3	99.67	695/704	41/704	32

U 的单位是百分比，接纳/用户 SLO 列都是通过数/704。active 是这一整秒都有已接纳工作（计算或 IO stall）的卡数，不是“恰好在窗口内到达过请求”的卡数。

$$U = \frac{\sum_i |\text{compute}_i \cap [1000, 2000]|}{32 \times 1000 \text{ ms}}.$$

逐卡还核对 1000 ms = compute + stall + idle。若 active < 32，低 U 不能全部归因 IO stall；报告保留真实空闲，不移动窗口。S1-S3 可重绑定，所以同一物理卡列未必处理同一请求；请求级对照必须按 request_id 匹配。

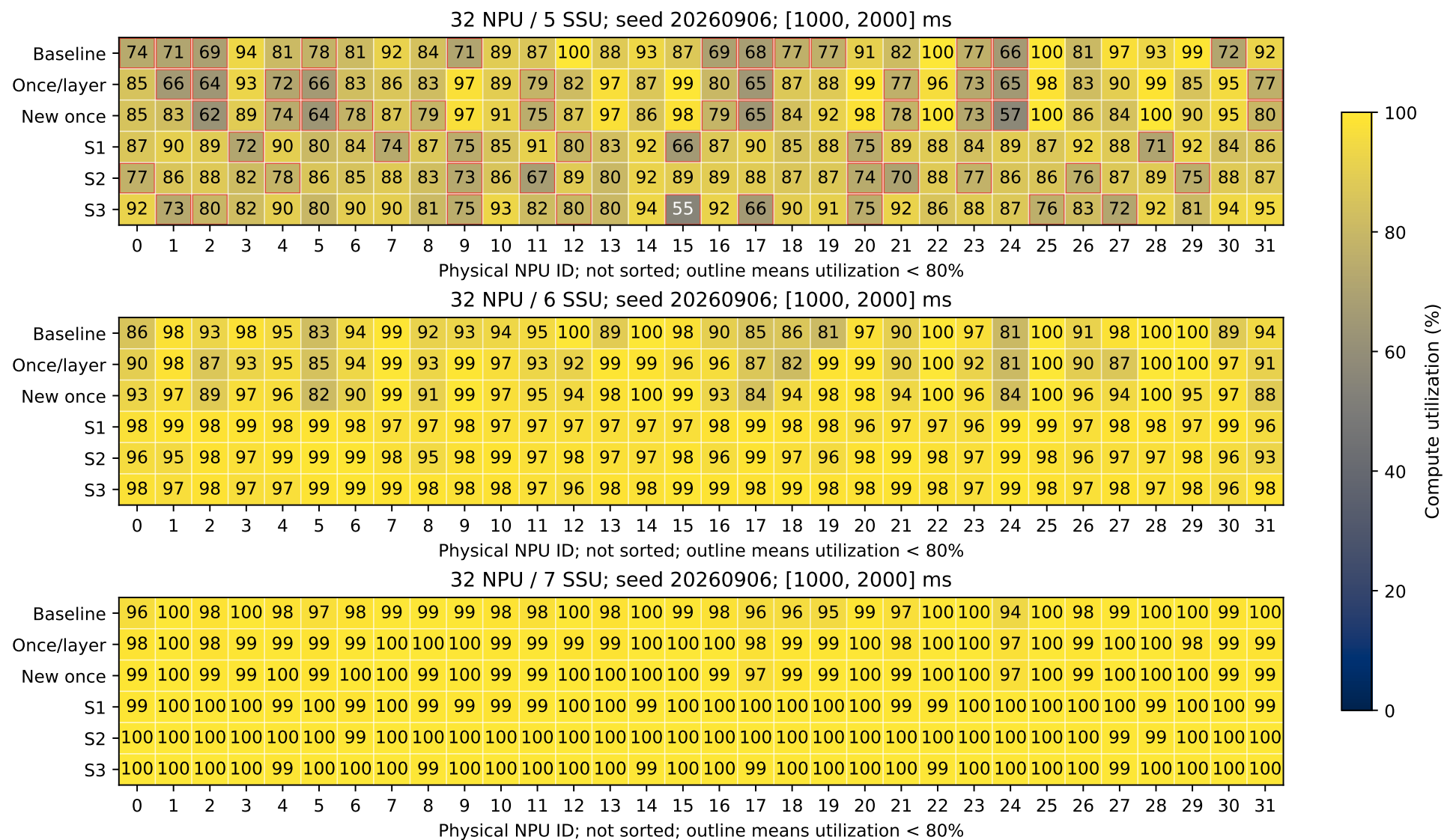


Figure 2A：种子 20260906。三个拓扑面板，每个六策略行，列始终为真实物理 NPU0-31。颜色统一 0-100%，格内四舍五入整数；100 不表示零 stall。红框按未舍入值低于 80% 判断，精确值见 per_npu_accounting.csv。没有逐行排序或跨种子平均卡号。

3 泛化检查：种子 20260907

SSU	策略	U/%	接纳 SLO	用户 SLO	active
5	Baseline	91.04	438/704	24/704	32
5	Once/layer	91.16	569/704	18/704	32
5	New once	91.13	559/704	25/704	32
5	S1	85.83	645/704	28/704	32
5	S2	86.04	650/704	30/704	32
5	S3	85.57	652/704	27/704	32
6	Baseline	98.73	558/704	24/704	32
6	Once/layer	98.22	667/704	27/704	32
6	New once	99.03	668/704	25/704	32
6	S1	98.23	693/704	36/704	32
6	S2	98.08	694/704	37/704	32
6	S3	98.83	692/704	38/704	32
7	Baseline	99.11	614/704	27/704	32
7	Once/layer	99.67	684/704	37/704	32
7	New once	99.68	682/704	39/704	32
7	S1	99.71	703/704	45/704	32
7	S2	99.78	703/704	46/704	32
7	S3	99.71	703/704	45/704	32

使用相同冻结策略参数；不因这一表出现退化再次调参。开发种子的单点提升不代表稳定提升。全部逐卡精确值、完整请求 SLO、各阶段控制计数另存 CSV/JSON，而不是只留有利案例。

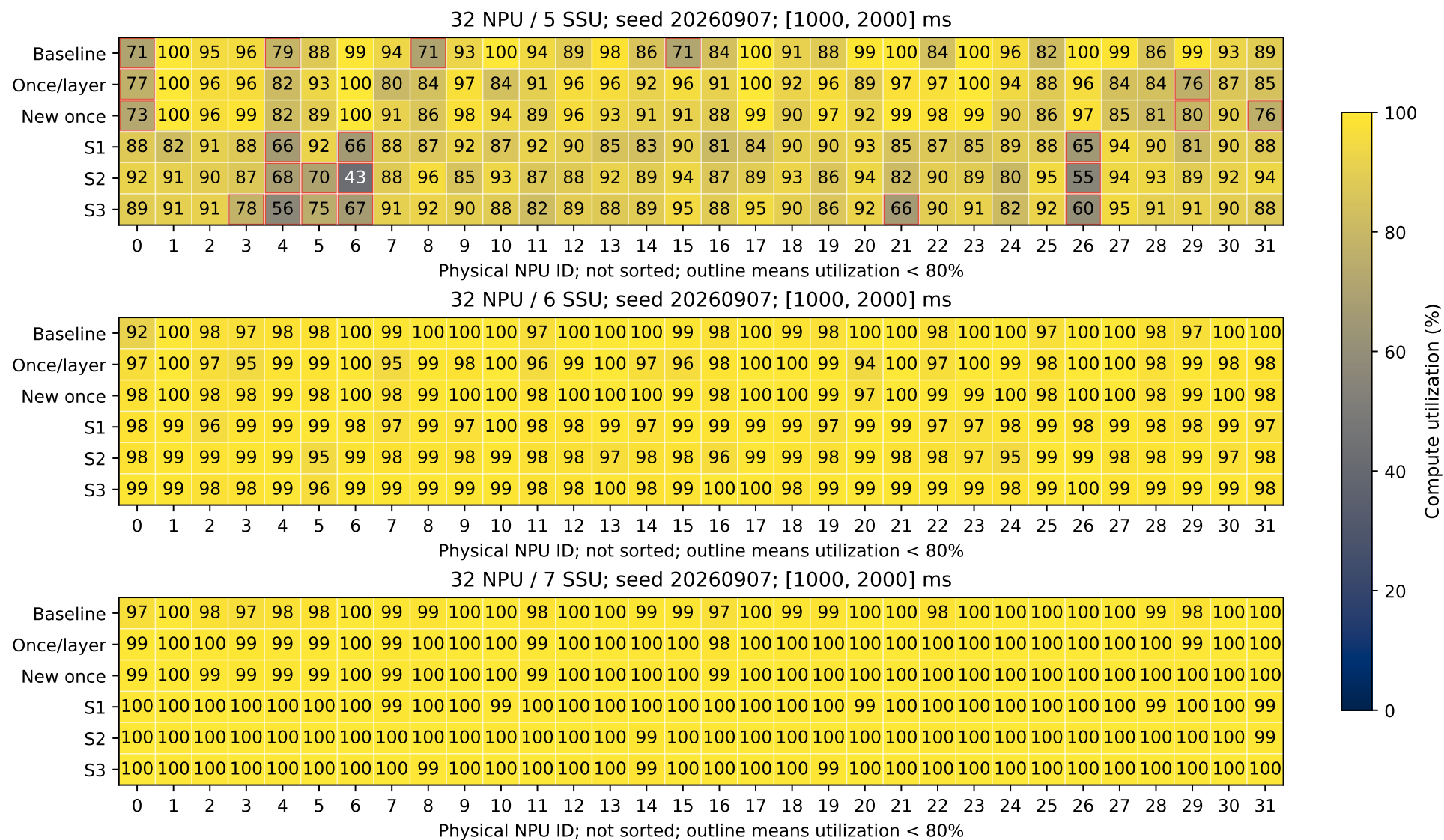


Figure 2B：种子 20260907，比例尺和真实卡号顺序与 Figure 2A 相同。100 仍为舍入值，不等于零 stall；红框按未舍入值判断，精确 CSV 另存。跨种子输入顺序不同，不先平均同卡值来掩盖差异。

4 如何选择参数，以及没选什么

策略	选卡	w/ms	joint	U/%	SLO 数	全程/ms	均尾等/ms
Baseline	—	—	—	93.26	555	4808.4	154.5
New once	—	—	—	94.64	668	4745.6	131.3
Once	—	—	—	93.74	663	4736.5	98.1
S1	fixed	0.25	—	95.95	687	4693.4	127.7
S1	compute	0.1	—	95.93	689	4878.8	299.3
S1	compute	0.25	—	96.15	691	5048.9	461.6
S1	compute	0.5	—	96.07	689	4902.8	315.5
S1	compute	1.0	—	96.01	687	5027.1	441.1
S3	compute	0.25	LS	96.23	685	4935.0	344.2
S3	compute	0.25	US	96.05	687	5093.9	510.0
S3	pipeline	0.25	LS	97.78	693	4746.6	219.2
S3	pipeline	1.0	LS	97.47	690	4791.9	247.8
S3	pipeline	0.25	US	96.91	691	4809.9	281.9
S3	pipeline	1.0	US	97.99	690	4837.4	296.3
S1	pipeline	0.25	—	97.12	693	4792.5	255.9
S1	pipeline	1.0	—	97.63	693	4807.6	280.8

此表只用 6 盘、种子 20260906，同一 canonical 输入。LS=least-slack, US=urgent-short。包括四种信用窗口、固定分卡消融、pipeline 选卡，以及 S3 与选卡/窗口的交互；开发候选是顺序扩展的工程搜索，不是预先注册的穷尽最优解。完整失败项保留在 data/development，修复前 pilot 不混入。

客户端冻结为 pipeline，w=1.0 ms，以用户指定的中间一秒 U 为首要目标；joint-rule 为 urgent short。这不是所有指标都最优。尤其 compute-only 选卡可能把短计算、重读取的请求集中到少数卡，增加全程末尾不均衡；必须同时看全程/ms 和均尾等，不能只按最高 U 删掉失败候选。算法可能减小每卡 IO stall，却把尾部等待变长。

正式开发组 S1 相对 Once：固定秒 U 改变 +3.891 个百分点，但完整 makespan 为 4807.610 对 4736.541 ms，即相对变化 +1.500%。正的 makespan 变化表示更慢，不因窗口 U 改善就称整批更快。原 Once 每卡全程计算范围 4405.605–4405.605 ms，选卡后为 4092.515–4708.062 ms；均尾等从 98.143 变为 280.828 ms。按逐卡四项恒等式，makespan 改变 +71.068 ms，由均 stall 改变 -111.617、均尾等改变 +182.685、其他闲改变 +0.000 ms 精确分解（显示数值四舍五入）。这是时间账目，不把排队与后续调度反馈全部归因为单一因素。

pipeline/w=0.25 ms 时，LS 与 US 的 U 分别为 97.78049% / 96.91409%，makespan 分别为 4746.565 / 4809.937 ms；pipeline/w=1.0 ms 时，LS 与 US 的 U 分别为 97.46877% / 97.98554%，makespan 分别为 4791.923 / 4837.398 ms。优先级规则与窗口存在交互；一组的排名不能外推为普遍最优。

逐步改变	U/%	较前项/pp	接纳 SLO 数	全程/ms
原 Once	93.7418	—	663	4736.541
加 Path 账本	94.6386	+0.8968	668	4745.579
再加全局发射	95.9491	+1.3105	687	4693.372
再后用选卡	97.1213	+1.1722	693	4792.547
信用窗口改 1 ms	97.6330	+0.5116	693	4807.610

这条链使用相同开发输入：New once→fixed 保留原 NPU 绑定；fixed→pipeline 保留 0.25 ms 窗口；最后只把 pipeline 的信用窗口变为 1 ms。它说明各次受控修改的闭环结果，不是可脱离上下文相加的普遍因果系数；后续层释放和队列状态随每次修改反馈变化。

5 输入究竟是什么

1 IO=1 个 128-token KV Block=176 KiB=180224 byte；每请求 8 层、batch=1。每 SSD 40 GiB/s，每卡只有一条总计 50 GiB/s 的接收链路。SSD→HBM，不经主机 DRAM 缓存。request_id 是请求唯一编号；layer 是 0–7；NPU ID 是 0–31；它们不是同一概念。

本文 coflow 特指“一个请求的一层所需的整组读取”。它按固定 placement 分成多个 SSD 分量，必须所有分量都到 HBM，这一层才能开始计算；只让其中一盘很快并不够。C 表示该请求一层的纯计算毫秒，不包含 IO 等待。每卡独立串行处理自己的请求，但各卡共享 SSD 服务能力；跨请求预取只针对已经真实到达的后继请求。

长度 K	NQL	IO/层	C/ms	原每卡配额
32	128	255	1.178026	4
32	256	254	1.997480	2
64	512	508	6.386487	2
96	512	764	9.070842	2
192	512	1532	17.123906	4
192	1024	1528	34.100782	3
192	2048	1520	68.056186	5

表中长度单位 K token；配额是原输入每卡的请求数，同一批次跨拓扑不改画像。所有实际每层有序 SSD placement 归档；选卡不能改变这些 IO 的盘归属。三个拓扑的逻辑 fingerprint 相同，同一拓扑六策略的完整物理输入 fingerprint 相同。

到达量	定义
初始积压	t=0 已到达 128 请求，原每卡 4 个
后续携带工作率	t>0 请求完整八层 GiB / 最后 arrival 秒
SSD 实际吞吐	真正已服务的字节 / 观测时长，与上项不同
满算需求	按原分卡 sum V / sum 8C，与 arrival 时钟不同

种子	盘	初始 GiB	后续 GiB	最后 arrival/ms	最热盘 GiB/s
06	5	189.209	772.604	3538.929	46.642
06	6	189.209	772.604	3538.929	39.785
06	7	189.209	772.604	3538.929	34.235
07	5	182.731	779.081	3568.600	46.518
07	6	182.731	779.081	3568.600	39.741
07	7	182.731	779.081	3568.600	34.231

后续携带工作率六组均为 218.315646777 GiB/s；表的“最热盘”也是后续携带工作率，不是实际 SSD 测量。初始工作只是这些已到达请求未来要读的八层总量，不是 t=0 一次发完。后续每个到达点都按累计工作量/固定速率核实验，不仅检验最后平均。

参考到达先规范到 1 ps，再按时间/request_id 排序。修复前浮点排序 pilot 仅保留在 exploratory_precanonical，不进入本报告。相同源码与种子本身不足以证明跨 Python 生成了完全相同的输入，必须检查 manifest。

6 三个新增策略究竟在哪一侧起作用

Baseline 固定每盘 Path0。Once 保留原独立选路纯函数。New once 保留原选路引擎，增加全局客户端 Path 账本。三者均不新选卡、不做新的盘内重排。

所有策略的每盘 256 条共享 Path 采用相同静态 CIR：四类总预算 20/6/8/6 GiB/s；PIR 无限，仍受整盘 40 GiB/s 上限。CIR 是保底而非硬上限，不能说 Baseline 被 Path0 很小的 CIR 锁死。运行期 CIR 写 0，满足写间隔至少 100 ms，但没有测试动态 CIR。原生模型明确不支持有限 PIR，本轮 PIR 均为无限。

6.1 S1：在排进 SSD 以前协调

到达选卡比较已到达的计算、接收链路和各盘均分服务负担： $score = \max(\text{剩余计算}, \text{单卡接收服务}, \text{最慢盘估计服务})$ ，加上新请求的全部已知八层工作后取最小。盘共享数量是对该盘有已知未完成工作的 NPU 数，不是未完请求数量，不读未来；该流体均分 score 不是实际完工时间预测。

所有卡共用全局发射器。已激活层提供 deadline、未发射各盘块数、尚未到 HBM 各盘块数。以 $b = 176/1048576$ GiB 表示一条 IO，估计

$$h = 1000b \max \left(\max_s K_s / 40, \sum_s K_s / 50 \right) \text{ ms.}$$

这里的 K 是客户端“未到 HBM”，可能已读完 SSD，故 h 只作优先级后发式，**不是当前 SSD 剩余时间的严格下界**。若 $deadline - now \leq h$ ，该 coflow 被视为紧急，紧急者优先短剩余，其他按 deadline。没有未来请求信息，也不把还没激活的所有后续层塞进当前 deadline。

信用窗口 $w = 1.0$ ms，单盘最多 $\max(1, \lfloor 40w / (1000b) \rfloor)$ 个已发送未 ACK IO；单卡上限将 40 换成 50。一批最多 64 个授权，实际发送消耗信用，HBM 回执归还；每次新层激活都会清空尚未执行的授权尾段并触发重规划，使新紧急 coflow 能参与。授权不是已经入队的 IO，不迁移盘内命令。

盘信用直到 HBM ACK 才归还：已离开 SSD 但滞留接收链路的 IO 仍占盘信用。因此信用限制并不保证 SSD 永远有工作可做，也不是盘真实 pending 数。

三套数必须区分：Path 账本是“已规划 - HBM ACK”，信用是“已实际发射 - HBM ACK”，coflow 剩余是“已激活层总数 - HBM ACK”。前两者不包含未规划未来层；它们都不是 SSD 精确实时队列。

6.2 S2：已入盘后，仅在原生等价 Path 中选

继承 S1，允许每盘在“最小 finish-tag+ 原生 EPS”候选中改变 Round Robin 选择，并重排选中 Path 的 pending 头。局部优先级用 deadline、该层在本盘的原始块数和入队时间。保持 Path 归属、CIR/PIR、tag 更新和正在服务 IO 不变。每次选择都断言 $selected-tag \leq min-tag + EPS$ ，没有超出原生等价集合；集合内允许 tag 略高于严格最小值。这不是无限制跨 Path EDF。

6.3 S3：同样的可选范围，用全局 coflow 进度键

S3 与客户端采用相同 urgent-short 键：紧急 coflow 先按剩余服务估计从短到长，未紧急的按 deadline；这不是最小 slack 规则。

该进度来自客户端已激活工作和 HBM ACK，经共同控制状态供各盘优先级使用，不是读取其他盘即时内部队列。每盘有自己的原生等价候选和忙碌状态，因此同一个共同键不保证所有盘同时服务同一 coflow。S2/S3 需要设备侧接口或固件配合，不能包装为普通 SSD 上仅改客户端就能部署。

S1-S3 到达选卡、发射函数完全相同，但不同历史服务会反馈到后续绑定和层激活。最终差值不能全部归因于某一次跨 Path 换序；同 Path/跨 Path 改变次数是机制证据，不是收益的独立因果分解。

7 为什么协调更多也未必消除等待

同请求预取下一层，预算是本请求当前层 C。跨请求 L0 则不同：只有前请求最后层开始时，才激活下一个已到达队首请求，预算是 **C_{previous}**，不是 **C_{next}**。

对于完整尚未读取的一层，孤立读取下界为上一节 h 的公式，但此处 K 必须是真实未读块数。它忽略首块流水启动、盘/链路竞争、QoS 和发射开销，故下界小于 C 不证明一定可隐藏。所有本轮画像的孤立下界都小于自身 C，仍不能排除跨请求边界预算不足。

实际实例来自 6 盘、种子 20260906、S3、NPU24。前请求 26000005 是 32K/NQL128，末层计算预算 1.178026 ms；后请求 11000006 是 96K/NQL512，自身每层 $C = 9.070842$ ms，但不能用这段尚未开始的计算隐藏它自己的 L0。

后请求 L0 共 764 IO (0.128235 GiB)，各盘块数依次为 [126, 128, 124, 122, 134, 130]。激活 = 1321.227376 ms，预算结束 = 1322.405402 ms，到齐 = 1323.796270 ms。孤立读取下界 2.564697 ms，对应最早到齐时刻的理论下界为 1323.792074 ms（不是实际事件），所以这次转换至少有 1.386671 ms 隐藏不了；实际 L0 stall = 1.390867 ms，其中窗口内 1.390867 ms。

为便于隔离预算原因，此例选实际等待最接近孤立下界的一次，不冒充最差卡或全体请求的典型值。证书只能说明至少这部分时间隐藏不了，不能把实际等待超过证书的剩余部分全归为固有容量不足。

这份证书只约束记录中的前后请求与激活时刻。不同选卡、顺序或允许更早多请求预取可能避免该相邻转换；它不是“任何策略 100% 都不可能”的全局证明。

更一般地，对固定当前状态和期限区间 $[a, D]$ ，若同盘必须在该区间读完的**真实 SSD 剩余字节**超过 $40(D - a) / 1000$ GiB，任何重排都不能全满足；同一 NPU 接收链路用 50。active IO 只能计未完成部分，已在链路的字节不重复计成 SSD 工作。未知 active 剩余可保守省略，而不能按全块充数。无超载证书只是未证伪，不等于已找到可行调度。

7.1 数学证书和原生仿真到底匹配什么

另外 16 个小型原生测试覆盖 4/32 卡与 5/6/7 盘，验证容量证书与 SSD→HBM 实际到齐时间的方向一致；不是把下界当精确预测器。它们只运行一层，并使用外部指定的读取 deadline；不改变正式八层 SLO。一般 native L0 deadline 是接纳时刻；这些 $t=0$ 接纳的小测试中该值为 0，不能把它当成测试另行指定的外部 deadline。

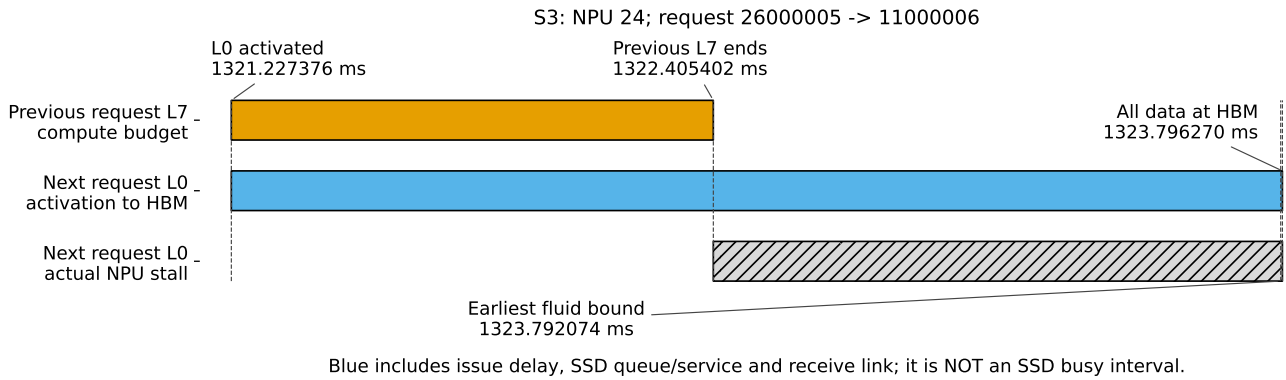


Figure 3：真实跨请求预算。橙色是前请求最后层计算；蓝色是后请求 L0 从激活到 HBM 到齐，包含发射等待、SSD 排队/服务和链路，不是 SSD 持续忙碌线；斜线为后请求实际 NPU stall。虚线标实际事件和明示的理论最早下界，后者不是实际完成事件；不用均匀取样刻度代替事件。

- 320 条 IO 固定在 SSD0，单盘至少要 1.342773 ms；给 1 ms deadline 时，证书中的请求至少一个实际迟到。其他盘空闲不能读取这批固定 placement。
- 每盘各 100 条发给同一 NPU，每盘本身可在 1 ms 内读完，但 5/6/7 盘合流可能超过这一卡 50 GiB/s 的接收能力；对应链路证书同样出现实际迟到。
- 反例一：只有 1 IO，流体下界约 0.004196 ms；deadline=0.005 ms 没有容量证书，但一条包必须先 SSD 服务再走链路，实际约 0.007553 ms，仍迟到。
- 反例二：200 条普通 IO 排在 1 条短 IO 前，同 Path FIFO 让小请求约 0.846786 ms 才到齐，错过外部 0.012 ms deadline；合法非抢占同 Path 换头后约 0.007553 ms 到齐。两者都没有容量证书，差异来自队序。仅此夹具的 client submit-batch-size=256 用来构造 FIFO 前驱；正式 client submit-batch-size=1，推理 batch-size 始终为 1。

因此有必要容量失败证书能证伪全部按时完成，没证书则仍可能因包流水或队序失约。策略启发式的成功率必须用实际仿真比较，不能声称公式“零误差预测所有 stall”。

8 全程 stall 与尾部不均衡：不能只看中间一秒

种子	盘	策略	全程/ms	全程 U/%	均 stall/ms	均尾等/ms	其他闲/ms
06	5	Baseline	5279.9	83.44	704.9	169.4	0.0
06	5	Once/layer	5389.1	81.75	719.0	264.5	0.0
06	5	New once	5279.4	83.45	724.8	149.0	0.0
06	5	S1	5785.1	76.15	640.4	739.1	0.0
06	5	S2	5586.5	78.86	658.5	522.4	0.0
06	5	S3	5477.1	80.44	628.9	442.6	0.0
06	6	Baseline	4808.4	91.62	248.3	154.5	0.0
06	6	Once/layer	4736.5	93.01	232.8	98.1	0.0
06	6	New once	4745.6	92.84	208.7	131.3	0.0
06	6	S1	4807.6	91.64	121.2	280.8	0.0
06	6	S2	4820.3	91.40	129.3	285.4	0.0
06	6	S3	4837.4	91.07	135.5	296.3	0.0
06	7	Baseline	4579.7	96.20	107.7	66.4	0.0
06	7	Once/layer	4549.3	96.84	97.6	46.1	0.0
06	7	New once	4535.8	97.13	82.0	48.2	0.0
06	7	S1	4779.8	92.17	52.0	322.3	0.0
06	7	S2	4777.8	92.21	48.3	323.9	0.0
06	7	S3	4773.6	92.29	47.9	320.0	0.0
07	5	Baseline	5402.5	81.55	727.3	269.7	0.0
07	5	Once/layer	5339.7	82.51	727.8	206.3	0.0
07	5	New once	5295.0	83.20	723.0	166.4	0.0
07	5	S1	5393.4	81.69	691.0	296.8	0.0
07	5	S2	5632.4	78.22	674.2	552.6	0.0
07	5	S3	5452.0	80.81	672.0	374.4	0.0
07	6	Baseline	4882.6	90.23	238.1	238.9	0.0
07	6	Once/layer	4807.1	91.65	235.8	165.7	0.0

种子	盘	策略	全程/ms	全程 U/%	均 stall/ms	均尾等/ms	其他闲/ms
07	6	New once	4774.6	92.27	207.4	161.6	0.0
07	6	S1	4920.3	89.54	187.9	326.8	0.0
07	6	S2	4844.8	90.93	164.7	274.5	0.0
07	6	S3	4859.3	90.66	166.5	287.2	0.0
07	7	Baseline	4667.3	94.39	126.2	135.5	0.0
07	7	Once/layer	4618.6	95.39	106.8	106.1	0.0
07	7	New once	4631.2	95.13	96.7	128.9	0.0
07	7	S1	4744.8	92.85	49.6	289.5	0.0
07	7	S2	4726.8	93.21	45.6	275.6	0.0
07	7	S3	4737.4	93.00	44.5	287.3	0.0

种子列 06/07 分别是两个完整种子尾号。“均 stall”是每卡全程 IO 等待毫秒；“均尾等”严格为全局 makespan 减该卡最后完工，再取平均；“其他闲”是首次/中途 idle，不偷算成末尾不均衡。

$$T_{\text{makespan}} = T_{\text{compute}} + T_{\text{stall}} + T_{\text{other idle}} + T_{\text{tail}} \quad (\text{逐卡}) .$$

完整 704 请求的计算总量相同，所以全程 U 随 makespan 反向变化；固定 [1,2] 秒 U 可能提高但 makespan 变长。SLO 按请求数计，U 按计算毫秒计，也可能方向不同。每卡四项精确值和最大尾差另存 data/per_npu_accounting.csv。

种子	盘	策略	均用户/ms	P99 用户/ms	最大尾等/ms
06	5	Baseline	1301.8	2347.9	358.9
06	5	Once/layer	1292.3	2351.5	517.8
06	5	New once	1287.4	2360.3	261.7
06	5	S1	1265.0	2007.2	1193.9
06	5	S2	1267.3	2022.3	1006.4
06	5	S3	1260.4	1979.8	1125.5
06	6	Baseline	1065.0	2018.8	287.7
06	6	Once/layer	1048.3	2003.3	223.0
06	6	New once	1032.3	1986.1	248.5
06	6	S1	975.1	1466.0	581.4
06	6	S2	974.1	1457.2	476.8
06	6	S3	978.9	1467.1	568.2
06	7	Baseline	977.8	1897.2	116.4
06	7	Once/layer	969.1	1904.2	103.2
06	7	New once	962.1	1903.1	93.1
06	7	S1	918.1	1422.9	515.8
06	7	S2	916.9	1421.3	493.1
06	7	S3	915.0	1423.6	528.5
07	5	Baseline	1260.5	2255.8	522.7
07	5	Once/layer	1254.5	2292.0	435.6
07	5	New once	1248.9	2215.7	351.6
07	5	S1	1214.4	1961.9	698.7
07	5	S2	1220.5	2097.5	963.4
07	5	S3	1213.8	1994.8	996.9
07	6	Baseline	995.4	1920.0	414.4
07	6	Once/layer	991.1	1957.8	312.3
07	6	New once	976.1	1933.7	293.6
07	6	S1	926.7	1417.0	536.8
07	6	S2	923.8	1386.0	588.2
07	6	S3	918.5	1390.7	524.0
07	7	Baseline	915.2	1894.0	221.7
07	7	Once/layer	907.9	1897.9	181.9
07	7	New once	899.1	1896.3	184.9
07	7	S1	827.1	1295.7	486.0
07	7	S2	825.7	1298.5	506.9
07	7	S3	826.1	1292.3	495.2

上表用户均值/P99 从 arrival 起算；最大尾等表示最早结束卡到全局完工的空隙，不是该卡的 IO stall。

9 控制代价与部署限制

盘	策略	路由/s	均路由/ms	全局规划/s	盘决策/s	模拟全程/s
5	Baseline	0.29	0.010	0.00	0.00	5.280
5	Once/layer	115.77	4.111	0.00	0.00	5.389
5	New once	165.87	5.890	0.00	0.00	5.279
5	S1	82.71	2.937	254.28	0.00	5.785
5	S2	93.49	3.320	271.56	132.47	5.587
5	S3	80.96	2.875	235.90	126.27	5.477
6	Baseline	0.21	0.006	0.00	0.00	4.808
6	Once/layer	167.27	4.950	0.00	0.00	4.737
6	New once	103.52	3.063	0.00	0.00	4.746
6	S1	124.30	3.678	270.72	0.00	4.808
6	S2	83.51	2.471	192.37	138.27	4.820
6	S3	81.78	2.420	185.14	143.50	4.837
7	Baseline	0.21	0.005	0.00	0.00	4.580
7	Once/layer	106.08	2.691	0.00	0.00	4.549
7	New once	99.82	2.532	0.00	0.00	4.536
7	S1	81.87	2.077	91.11	0.00	4.780
7	S2	80.63	2.045	91.51	138.54	4.778
7	S3	82.26	2.087	92.05	156.65	4.774

成本表为种子 06 的 Python 计时。路由一次是一个 request-layer-SSU 的 Path 规划；全局规划是最多 64 授权的一批；盘决策是一次原生候选集合选择。计时是并行实验机器上的 callback 墙钟，不是硬件 CPU 周期测量。完整两种子的细项，包括 collector、全局 ACK、Path ACK、enqueue 和次数，保存在 control_cost.csv。

这些时间没有进入上面的仿真时间。即使所有压力查询只读 5 ms 副本，也不等于决策免费。若累计路由秒数远大于模拟全程秒数，当前 Python 原型不能自动满足在线预算；需要增量结构、批量化、并行/本地化和实现语言优化后重新计时。计数器计时范围不是完整 profile，不把漏计部分视为 0。

Baseline/Once 的全局 Path ACK 维护仅用于本次守恒审计，不是它们选路所需的部署成本；New once/S1-S3 确实需要全 managed 流量的共享账本、一致性和消息传递。S3 还需要设备使用共同进度。网络延迟为 0、共同状态原子可见是额外理想假设，不从本地 Python ACK 时间推断分布式消息成本很低。

各策略同一个 collector 在 t=0 初始化，之后每 5 ms 每盘一次 fresh 读取，查询只有副本，不按 miss 偷读；CIR 运行写 0。S1-S3 另有全局每 IO 0.1 微秒的发射间隔，参考策略原约束是每 NPU 各自 0.1 微秒；前者更严格，但仍不是现实主机控制延迟模型。

10 函数、归档与复现

- shared_path_baseline.py: baseline_path_ids, 输入 IO 数, 输出 Path0 序列。
- shared_path_once.py: once_path_ids, 输入 IO 数、采集副本、允许 Path、QoS, 输出原 Once 序列。
- shared_path_new_once.py: new_once_path_ids, 另输入全局已规划未 HBM ACK Path 账本, 输出并由调用者原子预留。
- coflow_client_policy.py: choose_npu/choose_npu_pipeline 在真实到达时调用；plan_global_batch 输入已激活 ClientRead、各盘/卡实际 outstanding、当前时间, 输出有序授权。
- coflow_disk_policy.py: choose_path 输入本盘原生等价候选与 RR 游标, 输出选择；同 Path pending 换头由 adapter 配合。
- coflow_joint_policy.py: coflow_priority 输入固定 deadline、各盘已激活未 HBM ACK 块数和 now, 输出 S3 排序键。
- coflow_capacity_analysis.py: capacity_overloads 输入真实 remaining SSD/link 分开数与 release/deadline, 输出必要容量失败证书, 不输出“保证无 stall”。

S1 需要选卡、全局发射、New once 路由和 ACK 维护共同接入；单独调用 choose_npu 不等于运行完整 S1。S2/S3 还需要设备选择 hook。纯函数输出是授权、排序键或必要条件，不是精确未来完工时间。

选卡函数的 backlog/counter 数组必须按物理 NPU ID 0...N-1 排列，返回的数组索引才是 NPU ID。不能把按 U 排过序的列表传进去，再把索引当原物理卡号。各盘向量同样按 SSU ID 0...S-1 排列。

下列独立模块命令只是最小 demo，保留开发默认，并不自动复现正式配置。正式调用需显式指定：

- 选卡：choose_npu_pipeline，不是默认的 compute 选卡演示。
- 客户端发射：queue_window_ms=1.0。
- S3 纯函数：coflow_priority 的参数 rule="urgent_short"。
- adapter/runner：选卡参数 assignment="pipeline"，窗口参数 queue_window_ms=1.0，联合排序参数 joint_rule="urgent_short"。

纯函数参数叫 rule；adapter/runner 参数叫 joint_rule。完整正式命令位于 data/formal_results 目录下的 client_commands.json。

```
python -B shared_path_baseline.py
python -B shared_path_once.py
python -B shared_path_new_once.py
python -B coflow_client_policy.py
python -B coflow_disk_policy.py
python -B coflow_joint_policy.py
python -B coflow_capacity_analysis.py
python -B -m pytest -q test_coflow_client_policy.py test_coflow_disk_policy.py
python -B -m pytest -q test_coflow_disk_adapter.py test_coflow_sim_adapter.py
python -B -m pytest -q test_coflow_experiment_inputs.py test_coflow_report.py
python -B -m pytest -q test_coflow_capacity_analysis.py
python -B -m pytest -q test_coflow_capacity_simulation.py
```

完整再生成使用 build_coflow_report.py --data 指向 data/formal_results，加 --render。只有 36 组严格审计全部通过才生成最终 PDF。结果各自保存完整输入、实际 ordered placement 指纹和源文件 SHA 归档；不是用当前源码代替当时版本。report_audit.json 核对完成块数、逐层时间、两种 SLO、每盘字节、collector、信用归零和 native tag 权限。

源代码相同、同一机器/解释器仍不免除完整输入核验。正式环境记录在 formal_results/environment.json。输入/源 hash、单测与原生断言证明可复现和检查范围内的正确性，不证明任意真实 SSD 固件、用户分布或低延迟控制实现有相同收益。